PRACTICAL 9

MULTITHREADING AND SYNCHRONIZATION

Roll No.: 25IT037

Part A — Practice Programs

Aim: To understand race conditions in multithreaded Java programs and use synchronization to obtain correct results.

1. Counter Race

Aim: Create multiple threads that increment a shared counter. First run without synchronization and then use synchronization.

Program — Without Synchronization

```
class CounterRace {
    static int counter = 0;
    static void increment() { counter++; }

    public static void main(String[] args) throws Exception {
        Thread[] t = new Thread[10];

        for (int i = 0; i < 10; i++) {
            t[i] = new Thread(() -> {
                for (int j = 0; j < 100000; j++)
                    increment();
            });
            t[i].start();
        }

        for (Thread x : t) x.join();

        System.out.println("Counter = " + counter);
        System.out.println("Expected = 1000000");
    }
}
```

Output — Without Synchronization

```
Counter = 742381
Expected = 1000000
```

The wrong value can change on every run because several threads access the same variable simultaneously.

Program — With Synchronization

```
class CounterRaceSync {
    static int counter = 0;
    static synchronized void increment() { counter++; }

    public static void main(String[] args) throws Exception {
        Thread[] t = new Thread[10];

        for (int i = 0; i < 10; i++) {
            t[i] = new Thread(() -> {
                for (int j = 0; j < 100000; j++)
                    increment();
            });
            t[i].start();
        }

        for (Thread x : t) x.join();

        System.out.println("Counter = " + counter);
        System.out.println("Expected = 1000000");
    }
}
```

Output — With Synchronization

```
Counter = 1000000
Expected = 1000000
```

Result: Synchronization makes the shared counter update safe and the final value is exactly 1,000,000.

2. Seat Booking Race

Aim: Use 5 available seats and 10 threads trying to book one seat. Demonstrate overselling without synchronization and correct booking with synchronization.

Program — Without Synchronization

```
class SeatBookingRace {
    static int seatsLeft = 5;
    static int booked = 0;

    static void book() {
        if (seatsLeft > 0) {
            try { Thread.sleep(10); } catch (Exception e) {}
            seatsLeft--;
            booked++;
            System.out.println(Thread.currentThread().getName() + " booked a
seat");
        }
    }

    public static void main(String[] args) throws Exception {
        Thread[] t = new Thread[10];
```

```
        for (int i = 0; i < 10; i++) {
            t[i] = new Thread(SeatBookingRace::book, "Thread-" + (i + 1));
            t[i].start();
        }

        for (Thread x : t) x.join();

        System.out.println("Seats left = " + seatsLeft);
        System.out.println("Successful bookings = " + booked);
    }
}
```

Output — Without Synchronization

```
Thread-1 booked a seat
Thread-2 booked a seat
Thread-3 booked a seat
Thread-4 booked a seat
Thread-5 booked a seat
Thread-6 booked a seat
Thread-7 booked a seat
Seats left = 0
Successful bookings = 7
```

The output may change on different runs. More than 5 bookings can occur because the check and update are not protected.

Program — With Synchronization

```
class SeatBookingSync {
    static int seatsLeft = 5;
    static int booked = 0;

    static synchronized void book() {
        if (seatsLeft > 0) {
            seatsLeft--;
            booked++;
            System.out.println(Thread.currentThread().getName() + " booked a
seat");
        }
    }

    public static void main(String[] args) throws Exception {
        Thread[] t = new Thread[10];

        for (int i = 0; i < 10; i++) {
            t[i] = new Thread(SeatBookingSync::book, "Thread-" + (i + 1));
            t[i].start();
        }

        for (Thread x : t) x.join();

        System.out.println("Seats left = " + seatsLeft);
```

```
        System.out.println("Successful bookings = " + booked);
    }
}
```

Output — With Synchronization

```
Thread-1 booked a seat
Thread-2 booked a seat
Thread-3 booked a seat
Thread-4 booked a seat
Thread-5 booked a seat
Seats left = 0
Successful bookings = 5
```

Result: Exactly 5 bookings succeed and no overselling occurs.

3. Advanced Learners — Parallel Array Sum

Aim: Split a large array across several threads, demonstrate a wrong shared total, and fix it using synchronization.

Program

```
import java.util.Arrays;

class ParallelArraySum {
    static long total = 0;

    static void wrongSum(int[] a, int start, int end) {
        for (int i = start; i < end; i++)
            total += a[i];
    }

    static synchronized void add(long value) {
        total += value;
    }

    public static void main(String[] args) throws Exception {
        int n = 1000000;
        int[] a = new int[n];
        Arrays.fill(a, 1);

        Thread[] t = new Thread[4];

        total = 0;
        for (int k = 0; k < 4; k++) {
            int s = k * n / 4;
            int e = (k + 1) * n / 4;
            t[k] = new Thread(() -> wrongSum(a, s, e));
            t[k].start();
        }
        for (Thread x : t) x.join();

        System.out.println("Wrong total = " + total);
```

```
        total = 0;
        for (int k = 0; k < 4; k++) {
            int s = k * n / 4;
            int e = (k + 1) * n / 4;
            t[k] = new Thread(() -> {
                long part = 0;
                for (int i = s; i < e; i++) part += a[i];
                add(part);
            });
            t[k].start();
        }
        for (Thread x : t) x.join();

        System.out.println("Correct total = " + total);
        System.out.println("Expected total = " + n);
    }
}
```

Output

```
Wrong total = 684721
Correct total = 1000000
Expected total = 1000000
```

The wrong total can change because multiple threads update the shared total simultaneously.

Performance Measurement

Use System.nanoTime() before starting the threads and after all threads finish with join(). Run the program several times and compare the average execution time.

Conclusion

Multithreading can improve performance, but shared data can cause race conditions. The synchronized keyword provides safe access to shared resources and produces correct results.